Assignment 02 - Sensor Monitoring System

Embedded Application Design

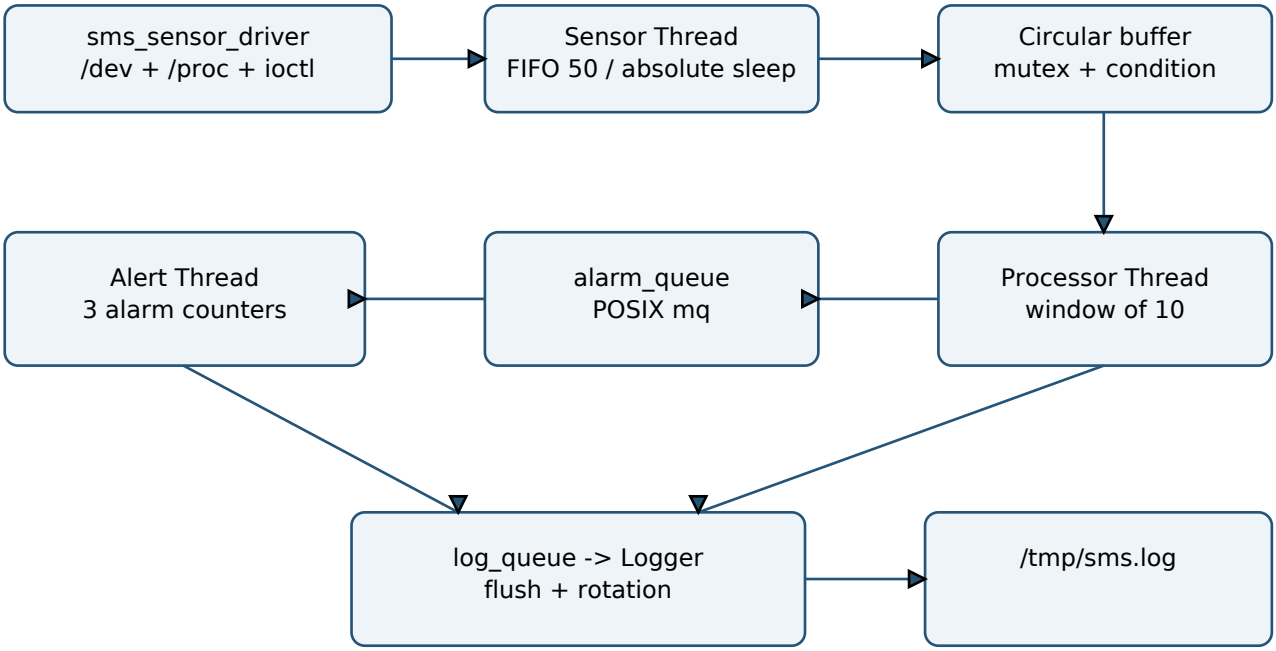
Mai Thế Tâm
Student ID: SE205068
Course: LOS201
Platform: ARMv7 Cortex-A9 in QEMU, Linux 5.15

This document describes a four-thread monitoring application connected to an implemented kernel sensor simulator. It covers data ownership, queueing, shutdown, memory, debugging, system-call behavior and measured wake-up jitter. It reports measured behavior separately for the ARM driver backend and the explicitly selected HOST simulator.

Deliverables include modular C source, ARM and x86_64 executables, the sensor module, a bootable initramfs, kernel configuration, executable scripts, and raw logs. Experimental values in this report are loaded from those logs by the report generator, rather than typed into the source as assumed results.

The scope is an emulated industrial-monitoring workload. Random measurements exercise the alarm logic; they do not model a physical sensor calibration, an actual fieldbus, or a certified hard real-time system. Conclusions about timing are restricted to this software, host load and virtual platform.

1. Architecture and data flow



The kernel exposes one CSV sample per offset-zero read. Sensor Thread is the sole owner of the device descriptor during operation. It parses the CSV, records wake-up error, and attempts a nonblocking push into the bounded circular buffer. Processor Thread owns the rolling window and publishes alarms and processed-sample messages.

Alert Thread consumes alarm messages, updates its private type counters and forwards human-readable messages to the log queue. Logger Thread alone owns the FILE pointer, formatting timestamps, flushing and rotating. This avoids concurrent stdio operations from time-critical threads.

The main thread coordinates lifecycle and joins the four workers. Five tasks therefore appear in /proc/PID/status. Diagnostic procfs reads are independent of application logging and do not consume additional sensor samples.

2. Sensor driver contract

The sensor driver registers a misc device named sms_sensor. The CSV record consists of monotonic timestamp milliseconds, temperature and humidity, followed by newline. Random temperatures are generated as integer tenths from 150 through 450; humidity uses tenths from 300 through 900. Integer formatting exports decimal values without executing floating-point instructions in kernel context.

A mutex covers the global interval, timestamp, last values, sample counter and proc snapshot. Each opened file description has a separate bounded CSV snapshot. A successful offset-zero generation increments read_count once; subsequent partial reads consume that snapshot, and a read at its end returns EOF. Userspace seeks back to offset zero for the next sample.

SMS_SET_INTERVAL accepts an unsigned integer pointer, rejects zero and values above 60000 ms, and returns ENOTTY for unknown requests. The driver returns EAGAIN if the minimum interval has not elapsed. The application treats EAGAIN, EINTR and EOF as retryable conditions instead of corrupting a sample or silently terminating.

The assignment names ktime_get_ms(), but Linux 5.15 has no helper with that name. This implementation uses ktime_to_ms(ktime_get()) to obtain the same monotonic-millisecond quantity. Uptime uses div_u64 because native 64-bit division in an ARM32 module can otherwise generate an unresolved __aeabi_uldivmod reference.

Initialization creates the misc registration then proc entry. Failure to create proc deregisters misc. Exit removes both. The module owner pins callbacks while device descriptors remain open; forced removal is not used.

3. Application lifecycle and configuration

State	Entry / exit conditions	Ownership
INIT	Validate CLI, allocate, open driver, ioctl, create queues, lock memory	Main owns all resources
RUNNING	Create Logger, Alert, Processor, Sensor; sample until duration, sample limit, signal or error	Workers own their assigned data
SHUTDOWN	Stop producer; close buffer; drain processor, alarm queue and log queue; join workers	Main reclaims after joins

Defaults are interval 500 ms, temperature threshold 40 degrees Celsius, humidity threshold 80 percent, duration 60 seconds and Sensor FIFO priority 50. Flags -i, -t, -h, -d and -v follow the assignment. Additional explicit switches select OTHER scheduling, the HOST simulator, a sample limit, and configurable device/log/jitter paths.

CLI parsing rejects missing or malformed numbers, negative unsigned settings, zero intervals, nonfinite thresholds, and unexpected positional arguments. Opening a missing real device returns a nonzero exit status. Failing FIFO scheduling or mlockall is reported as an error for an RT run, not silently replaced by OTHER.

SIGINT and SIGTERM handlers set only volatile sig_atomic_t interrupted. They never lock, allocate, print, or call message-queue functions. Main observes the flag and initiates the normal shutdown path. Runtime errors set atomic stop and failed flags. A normal finite run or clean SIGINT returns zero; initialization/runtime failures return nonzero.

4. Circular buffer and rolling statistics

The buffer allocates 64 SensorData entries once on the heap. It maintains head, tail, current size, closed status, produced, consumed and overflow counters under one mutex. Sensor does not block when the ring is full: it increments overflow and emits a warning through the logging mechanism. This policy favors sampling cadence over guaranteed delivery under overload.

Processor waits on a condition variable while size is zero and the buffer is not closed. The wait is inside a while loop so spurious wake-ups cannot cause a read from an empty ring. Closing the buffer broadcasts the condition; Processor drains accepted entries before returning. The getter copies all exposed statistics under the same mutex.

A ring has predictable memory use and O(1) push/pop with no per-sample malloc. A linked list would introduce allocation failures, fragmentation and less predictable cache behavior. The capacity is a burst allowance, not an unlimited guarantee against a stalled consumer.

Processor privately owns a ten-element rolling window, index, sample count and temperature/humidity sums. Before the window fills, averages divide by the actual available count. Once full, the oldest contribution is subtracted before the new value is inserted. These variables require no mutex because no other thread accesses them.

Alarms use the current measurement for high-temperature/high-humidity checks and compare consecutive raw temperatures for a spike greater than 5 degrees. Rolling averages are logged for analysis, rather than delaying a threshold alarm until ten samples accumulate. The first sample has no predecessor, so no spike alarm is generated for it.

5. POSIX queues and backpressure

Two POSIX message queues separate alarm delivery from log delivery. Names include the PID and are created with O_CREAT|O_EXCL and mode 0600. Each queue is immediately unlinked after successful creation: open descriptors remain valid, but a crash will not leave a named queue requiring manual removal. Attributes request ten messages, within the standard Linux msg_max default.

Alarm messages contain a timestamp, integer type, explicit reserved field, value and threshold. The explicit reserved field replaces implicit compiler padding and is initialized to zero. This matters because mq_send transfers the entire structure representation, including padding bytes, not merely its logical members.

Timed queue operations block during normal empty/full conditions and periodically check lifecycle flags. They avoid permanent waits after a peer has failed or after a producer has finished. Timeout is an expected control event, not itself an application error. Unexpected queue errors set the failure status.

No queue send/receive occurs while holding the circular-buffer mutex. Processor releases the buffer lock before formatting or sending a message. This prevents a full queue from holding up the producer through an unrelated lock. Backpressure can still slow processing, which is measured via overflow_count rather than hidden by unbounded allocation.

POSIX mq fits small typed alarms and provides blocking notification and message boundaries. Shared memory would avoid a kernel copy but would require a separate notification protocol, ownership rules and queue implementation. For this low-rate workload, the simpler explicit message semantics outweigh that potential optimization.

6. Synchronization and shutdown proof

Shared object	Protection / owner	Observation point
Ring entries and counters	buffer.mutex + condition variable	buffer_stats snapshot
stop and done flags; failure	C11 atomic operations	All worker loops
Signal notification	volatile sig_atomic_t	Main polling loop
Sensor fd and jitter array	Sensor-only during run	Main after join
Rolling window	Processor-only	Processed log messages
Alarm type counts	Alert-only during run	Main after join
Log FILE and rotation	Logger-only	Files after close
Driver global state	sensor_lock mutex	proc snapshot

There is only one application data mutex, and it is never held across a queue call or a join. Condition waits atomically release that mutex while sleeping. Consequently the application lock graph has no nested-lock cycle. Queue dependencies flow Sensor -> Processor -> Alert/Logger, with no consumer waiting for a resource held by an upstream producer.

Main joins in reverse creation order: Sensor, Processor, Alert, Logger. Sensor closes the ring. Processor exits after draining it and sets processor_done. Alert drains alarms and exits on a subsequent empty timeout. Logger drains logs after alert_done, then closes the file. No producer can enqueue a later log once that completion condition is established.

Partial initialization also sets completion flags for workers that were never created. This lets already-started consumers terminate. Runtime logger failure sets logger_done so app_log fails promptly instead of blocking forever. Alarm publication uses timed sends and checks alert_done for the same reason. This is a bounded shutdown protocol, not cancellation of arbitrary lock-owning threads.

7. Logging, file ownership and validation

Logger receives messages from both Processor and Alert, prefixes event messages with local realtime date/time and milliseconds, and appends to the configured path. Sensor start/overflow/retry and worker startup messages make the execution paths visible. Logger itself writes an initial startup marker.

Flush is checked against CLOCK_MONOTONIC approximately every five seconds, using receive timeouts to wake even without messages. fclose performs the final flush. Rotation checks the logical FILE position after each line: above 1 MiB, it closes the file, renames it to the .1 path and opens a new file. One previous generation is retained, bounding disk growth.

Only Logger manipulates the file. Sensor and Processor therefore avoid acquiring a file lock or flushing per sample. A logging I/O error stops the application and causes a nonzero return rather than reporting a clean run with missing evidence.

```
invalid ['-i', '0']: exit=2
invalid ['-i', '-1']: exit=2
invalid ['-t', 'nan']: exit=2
invalid ['-d', 'bad']: exit=2
invalid ['extra']: exit=2
invalid ['--other', '--device', '/missing']: exit=1
rotation over 1 MiB: PASS
```

The rotation test preloads a temporary log above the threshold, runs the application, checks a .1 generation exists and verifies the current file is below the rotation threshold. It uses an isolated temporary directory, preventing accidental replacement of a real user log.

The default device and log paths are macros, with runtime override options. /tmp logs are intentionally ephemeral in the guest. Measurement scripts copy relevant outputs out before shutting down QEMU; the repository logs are therefore durable evidence rather than assumptions about guest persistence.

8. ARM integration and memory footprint

The ARM validation run used the real /dev/sms_sensor backend at 500 ms for 60 seconds with FIFO scheduling. proc read_count was recorded before and after the run; memory was sampled from the same process at the requested times. Main plus four worker tasks remained present.

Time	VmSize	VmRSS	VmData	Threads
5 s	4020 kB	2748 kB	1740 kB	5
30 s	4020 kB	2748 kB	1740 kB	5
55 s	4020 kB	2748 kB	1740 kB	5

```
read_count: 0
last_temperature: 0.0
last_humidity: 0.0
uptime_seconds: 0
interval_ms: 100

read_count: 118
last_temperature: 29.1
last_humidity: 69.6
uptime_seconds: 60
interval_ms: 500
```

VmSize and RSS are unchanged across these snapshots. This is consistent with one-time ring/jitter allocation, fixed worker stacks and bounded queues. It is not a proof against every possible long-running leak; the independent Valgrind/SIGINT test checks dynamic allocation lifetime on HOST.

A minimum interval applies at the driver as well as the application. Wake-up error can make a nominally periodic read arrive slightly earlier than the minimum since the last successful read; such a read receives EAGAIN and is retried at the next period. Therefore jitter wake-up count, produced sample count and driver read_count must be distinguished. The recorded 60-second result lies within the assignment tolerance of about 120 plus or minus ten.

9. Address-space regions and allocations

```
004c5000-004ca000 r-xp 00000000 00:02 86         /bin/sms_app
004ca000-004cb000 r--p 00004000 00:02 86         /bin/sms_app
004cb000-004cc000 rw-p 00005000 00:02 86         /bin/sms_app
004cc000-004ed000 rw-p 00000000 00:00 0          [heap]
76bbf000-76bc0000 ---p 00000000 00:00 0
76bc0000-76c21000 rw-p 00000000 00:00 0
76c21000-76d00000 ---p 00000000 00:00 0
76d95000-76d96000 ---p 00000000 00:00 0
76d96000-76dd6000 rw-p 00000000 00:00 0
76dd6000-76dd7000 ---p 00000000 00:00 0
76dd7000-76e17000 rw-p 00000000 00:00 0
76e17000-76e18000 ---p 00000000 00:00 0
76e18000-76eba000 rw-p 00000000 00:00 0
76eba000-76fc7000 r-xp 00000000 00:02 112        /lib/libc.so.6
76fc7000-76fc9000 r--p 0010c000 00:02 112        /lib/libc.so.6
76fc9000-76fca000 rw-p 0010e000 00:02 112        /lib/libc.so.6
76fca000-76fd6000 rw-p 00000000 00:00 0
76fd6000-76ff2000 r-xp 00000000 00:02 111        /lib/ld-linux-armhf.so.3
76ff2000-76ff4000 r--p 0001b000 00:02 111        /lib/ld-linux-armhf.so.3
76ff4000-76ff5000 rw-p 0001d000 00:02 111        /lib/ld-linux-armhf.so.3
```

The executable r-x mapping contains application machine code; r-- and rw- mappings contain read-only data, writable globals and relocation-related data. The [heap] region backs malloc allocations such as the ring and jitter array. Private anonymous mappings also serve libc allocator arenas and thread-related storage.

Each worker requests a 256 KiB pthread stack. Anonymous rw- ranges adjacent to inaccessible --- guard pages are consistent with those stacks; libc can merge stack/arena mappings, and maps alone does not reliably assign each anonymous range to a particular worker name. Main has the [stack] region. The full maps capture is included, rather than claiming every anonymous mapping is a stack.

libc.so.6 executable/read-only/writable mappings provide system-call wrappers, pthreads, queues, malloc and stdio on this glibc version. ld-linux-armhf.so.3 is the dynamic linker. The final kernel-provided vectors/vvar or vdso mappings, when shown, support userspace transitions/time access. Shared-library mappings and heap allocations are distinct from the fixed-size SensorData ring.

The jitter array reserves 100000 long entries once. On ARM32 this is approximately 400 KiB; on x86_64 it is approximately 800 KiB. The allocation is capped even for infinite duration; the report explicitly records the number of samples retained. No per-sample heap allocation grows with runtime.

10. Valgrind and a real memory defect

```
==24616== Thread 4:
==24616== Syscall param mq_timedsend(msg_ptr) points to uninitialised byte(s)
==24616==    at 0x491889C: mq_timedsend@@GLIBC_2.34 (mq_timedsend.c:35)
==24616==    by 0x10C957: detect_alert (processor_thread.c:21)
==24616==    by 0x10CBA8: processor_thread (processor_thread.c:40)
==24616==    by 0x490CB83: start_thread (pthread_create.c:447)
==24616==    by 0x4999CC3: clone (clone.S:100)
==24616==  Address 0x4f47cbc is on thread 4's stack
==24616==  in frame #1, created by detect_alert (processor_thread.c:10)
==24616==
==24616==
```

The initial run had no leaked heap blocks, but Valgrind detected uninitialized bytes passed to mq_timedsend from detect_alert. The cause was an implicit padding gap in the binary Alarm structure. Initializing visible fields through a compound literal did not make the padding representation initialized for the system call.

The repair adds an explicit reserved integer in the gap, initializes it, and initializes the message array. This produces a fully defined 32-byte message layout on both tested targets. Treating only leak totals as success would have missed this defect.

```
==38828==
==38828== HEAP SUMMARY:
==38828==     in use at exit: 0 bytes in 0 blocks
==38828==   total heap usage: 19 allocs, 19 frees, 818,555 bytes allocated
==38828==
==38828== All heap blocks were freed -- no leaks are possible
==38828==
==38828== For lists of detected and suppressed errors, rerun with: -s
==38828== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

The final HOST run lasted at least 31 seconds before SIGINT, selected the explicit simulator and OTHER scheduling, joined workers and freed all heap blocks. Valgrind reports zero errors. This validates userspace memory behavior; it neither checks kernel allocations nor proves FIFO timing under emulation. Source snapshots and the before/after logs permit inspection of the correction.

11. GDB: multi-thread alert injection

The reproducible batch script selects the alert-logic scenario. It breaks in detect_alert, lists threads and requests thread apply all bt. With scheduler-locking enabled for stepping, it changes the local sample temperature to 45.0, steps to the local temperature assignment and inspects it. Scheduler locking is disabled before continuing so the Alert consumer can run.

```
Breakpoint 1 at 0x46e5: file processor_thread.c, line 10.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
STATE INIT -> RUNNING
[New Thread 0x7ffff7edd6c0 (LWP 32124)]
[New Thread 0x7ffff7e9c6c0 (LWP 32125)]
[New Thread 0x7ffff7e5b6c0 (LWP 32126)]
[New Thread 0x7ffff7bff6c0 (LWP 32127)]
SENSOR policy=OTHER priority=0 backend=HOST_SIMULATOR
[Switching to Thread 0x7ffff7e5b6c0 (LWP 32126)]
Thread 4 "sms_app_x86" hit Breakpoint 1, detect_alert (a=0x7fffffffd870, sample=...,
previous=0, have_previous=false) at processor_thread.c:10
10      {
  Id   Target Id                                         Frame
  1    Thread 0x7ffff7fa2740 (LWP 32103) "sms_app_x86" 0x00007ffff7cecbdf in
__GI___clock_nanosleep (clock_id=clock_id@entry=0, flags=flags@entry=0,
req=0x7fffffffd690, rem=0x0) at ../sysdeps/unix/sysv/linux/clock_nanosleep.c:78
... full backtraces: logs/gdb_sms.txt ...
$2 = {timestamp = 819591, temperature = 45, humidity = 71.200000000000003}
[ALERT][819591] TYPE: TEMP_HIGH VALUE: 45.0 THRESHOLD: 40.0
```

The modified value is a debugger injection, not a naturally measured sensor reading. The captured stderr later shows TEMP_HIGH with VALUE 45.0 and THRESHOLD 40.0. The script disables the breakpoint before continuation and the process exits normally after the finite run.

The full session includes call stacks for the sleeping Sensor, condition/queue-waiting consumers, and the main coordinator. Waiting consumers are expected in a low-rate pipeline and are not evidence of deadlock. The final summary and successful process exit demonstrate the pipeline resumed after debugger intervention.

12. strace and CPU versus waiting time

Top by call count	Calls	% traced syscall time
mq_timedreceive	1077	33.96
futex	1044	17.56
mq_timedsend	921	2.96
clock_nanosleep	898	41.79
lseek	614	1.15

The capture uses strace -f -c for 30 seconds so all worker threads are traced. mq_timedreceive corresponds to blocking alarm/log consumers; futex underlies mutex/condition operations and thread joining; clock_nanosleep drives both absolute sensor periods and main polling. mq_timedsend carries processed samples, alarms and formatted log messages. Writes include stderr alarm output and buffered file writes.

The HOST backend is explicitly simulated: its syscall mix does not include real sensor-device reads. Calls to lseek can also come from stdio file-position queries for rotation. Error totals include expected receive timeouts and futex races; they are not all fatal application errors.

```
        Command being timed: "outputs/embedded-linux-labs/assignment02/app/sms_app_x86
--simulate --other -i 100 -d 30 --log /tmp/se205068-time-sms.log"
        User time (seconds): 0.01
        System time (seconds): 0.02
        Percent of CPU this job got: 0%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:30.10
        Average shared text size (kbytes): 0
```

strace percentages partition measured syscall time, including time spent blocked, and cannot be used as user-versus-kernel CPU percentages. The separate time command provides user and system CPU seconds. Both are small compared with approximately 30 seconds elapsed: this configuration is dominated by waiting for periods and IPC, rather than sustained CPU computation. HOST and emulated ARM timing must not be conflated.

13. perf: scheduler and IPC cost

The HOST perf capture uses cpu-clock sampling at 997 Hz with call chains, a 10 ms simulator interval and a 30-second run. This higher rate makes pipeline activity visible without claiming that it is the same workload as the required 500 ms ARM validation run. The report uses self overhead with children excluded.

```
8.99% sms_app_x86 [kernel.kallsyms] [k] _raw_spin_unlock_irqrestore
7.91% sms_app_x86 [kernel.kallsyms] [k] finish_task_switch.isra.0
3.60% sms_app_x86 libc.so.6 [.] __printf_buffer
3.60% sms_app_x86 libc.so.6 [.] clock_nanosleep@GLIBC_2.2.5
3.42% sms_app_x86 libc.so.6 [.] __printf_fp_buffer_1.isra.0
2.52% sms_app_x86 [kernel.kallsyms] [k] do_mq_timedsend
2.52% sms_app_x86 libc.so.6 [.] __GI___pthread_disable_asynccancel
2.52% sms_app_x86 libc.so.6 [.] mq_timedreceive@@GLIBC_2.34
```

The leading captured symbol is a kernel wake-up/spin-unlock path rather than the rolling-average arithmetic. Its call chain includes message-queue sending and waking consumers. That is plausible for a small pipeline whose work per sample is low and whose coordination crosses kernel queues repeatedly.

A concrete optimization would batch routine processed-sample log messages and reduce frequent main-loop polling, while leaving urgent alarms unbatched. A preallocated in-process log ring with a condition variable could reduce mq syscalls, but would require its own bounded ownership protocol and would change the current chosen architecture. Any change should be measured against both wake-up jitter and log latency.

The perf recording was taken before the padding-only IPC repair; its raw perf.data and text report are retained. The repair initializes the same-size message representation and does not change alarm decisions. Thus the profile is an explicitly labeled development snapshot, not falsely presented as a fresh recording of the final binary. Zero lost samples is reported in the capture header; finite sample counts still limit statistical precision.

14. Real-time implementation and results

Sensor uses SCHED_FIFO priority 50 in the default run and SCHED_OTHER only when explicitly requested. It calls clock_nanosleep with CLOCK_MONOTONIC and TIMER_ABSTIME, advancing a fixed expected deadline rather than sleeping relative to the previous completion. This avoids cumulative processing-time drift. mlockall(MCL_CURRENT|MCL_FUTURE) is requested before worker creation; errors are visible.

Each measurement records actual wake-up minus expected wake-up in microseconds. Both comparison runs use the real ARM driver, interval 100 ms and 1000 wake-ups each. This interval reduces experiment duration while the separate 60-second validation tests the default 500 ms configuration.

Policy	N	Min us	Avg us	Max us	P99 us
FIFO	1000	168	361.003	5571	2679
OTHER	1000	168	472.038	6085	3183

```
backend=ARM_DRIVER policy=FIFO interval_ms=100 samples=1000
min_us=168 avg_us=361.003 max_us=5571 p99_us=2679
[0,100)us=0 [100,500)us=936 [500,1000)us=37 [1000,+inf)us=27

backend=ARM_DRIVER policy=OTHER interval_ms=100 samples=1000
min_us=168 avg_us=472.038 max_us=6085 p99_us=3183
[0,100)us=0 [100,500)us=856 [500,1000)us=101 [1000,+inf)us=43
```

Using (OTHER P99 - FIFO P99) / OTHER P99, the measured P99 change is 15.83 percent. A negative value means FIFO was worse in this finite comparison. Scheduling class alone cannot guarantee an improvement under virtual-machine scheduling and changing HOST activity. No result is rounded into a claim of hard real-time behavior.

15. Timing interpretation and improvements

A monitoring design must choose a timing budget from its actual process requirements. As an illustrative engineering budget for a 500 ms reporting sensor, 5 ms wake-up error is one percent of the sampling period. That is an explicit design assumption, not an industrial standard or a universal safe bound. Fast control or protection systems can need much tighter deadlines.

The measured worst wake-up errors are 5571 us for FIFO and 6085 us for OTHER in the 100 ms experiments. Both observed maxima exceed the illustrative 5000 us budget, so this experiment does not satisfy that chosen maximum-error criterion. A sampled maximum is only the largest event observed, not a proven worst-case execution or scheduling bound. The 500 ms run has its own raw jitter report.

PREEMPT_RT can reduce nonpreemptible kernel sections and change interrupt handling, but it does not remove HOST scheduling delays from QEMU. CPU isolation, controlled IRQ affinity, pinning related workloads, priority-inheritance mutexes, and minimizing work in Sensor are candidates for a physical target. Each adds operational constraints and requires measurement under representative load.

The current mutex is short-lived and no queue calls are performed while it is held. Still, FIFO priority does not imply priority inheritance for a default pthread mutex. A lower-priority Processor holding the ring lock can delay Sensor; configuring PTHREAD_PRIO_INHERIT and measuring contention would be a focused follow-up if the latency budget demands it.

Missing wake-ups are not made invisible: the application advances absolute deadlines and records errors, and the driver enforces its own minimum interval. A late wake-up followed by a catch-up can cause EAGAIN. Improving that behavior would require an explicit policy for skipping missed periods or matching driver interval to acceptable early-arrival tolerance, with corresponding changes to sample-count expectations.

16. Reproduction, limits and references

```
bash assignment_support/scripts/run-qemu.sh 2
# In QEMU:
test_arm.sh
# Compare 1000 wake-ups per policy:
sms_app -i 100 -d 0 --samples 1000 --jitter /tmp/fifo.txt
sms_app -i 100 -d 0 --samples 1000 --other --jitter /tmp/other.txt
```

The kernel configuration enables POSIX_MQUEUE and HIGH_RES_TIMERS; the original LAB01 configuration did not. The initramfs mounts proc, sysfs, devtmpfs, tmpfs and mqueue. ARM libc and loader are carried forward from the LAB03 tool image. Build with make host / make arm, or use the shared support script with KDIR pointing at the built ARM Linux tree.

Verification scope: kernel-module build and ARM integration, 60-second execution, proc sample-count checks, three memory snapshots, clean SIGINT, HOST Valgrind, GDB alert injection, strace, perf, rotation and invalid-argument tests, and two real ARM jitter runs. The collected logs are retained even when they show a defect or a scheduling result less favorable than expected.

No physical sensor, persistent flash deployment, PREEMPT_RT comparison kernel, network publication or hard deadline guarantee is claimed. HOST simulator profiles exercise application/IPC paths and exclude the kernel sensor implementation. Reproducing numerical values exactly is not expected because random samples, HOST activity and scheduler behavior vary.

Primary references: Linux 5.15 kernel source and Documentation/driver-api/ioctl.rst; https://docs.kernel.org/driver-api/ioctl.html ; https://man7.org/linux/man-pages/man7/mq_overview.7.html ; https://man7.org/linux/man-pages/man2/clock_nanosleep.2.html ; https://man7.org/linux/man-pages/man2/mlockall.2.html ; https://man7.org/linux/man-pages/man7/sched.7.html ; https://man7.org/linux/man-pages/man7/pthreads.7.html . API documentation informed the implementation; experimental data comes from the accompanying logs.